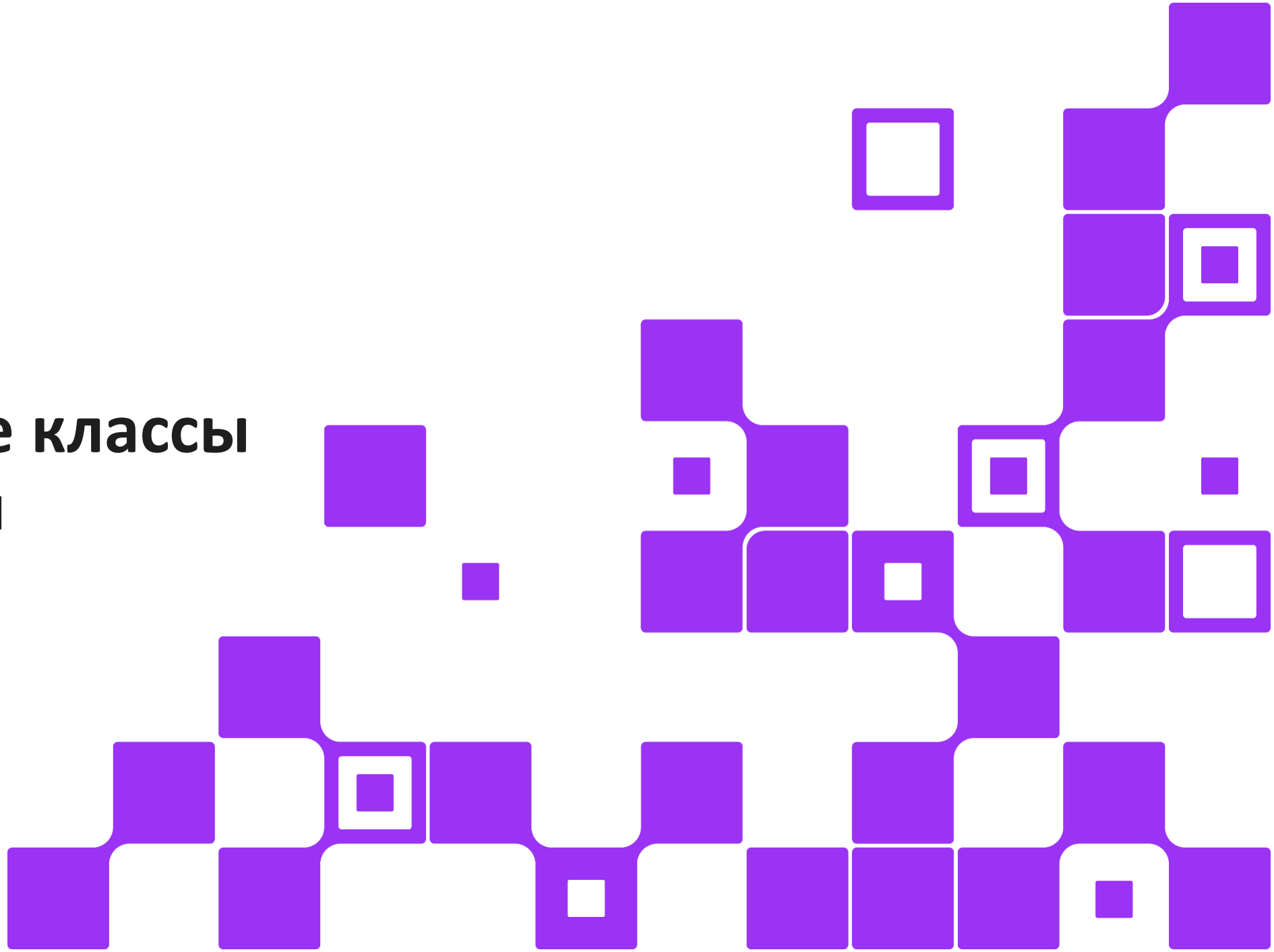
# Абстрактные классы
# Интерфейсы



IThub

В Java, абстрактные классы и интерфейсы являются ключевыми инструментами для организации объектно-ориентированного кода. Они оба помогают в разработке гибкого, расширяемого и удобочитаемого кода.

**Абстрактный класс** – это класс, который определяет общую структуру для своих подклассов, но не может быть использован для создания объектов.

**Интерфейс** – это набор методов, которые должны быть реализованы классами, которые реализуют интерфейс. Интерфейсы определяют только методы, но не содержат реализации.

# Абстрактные классы

- когда желательно определить суперкласс, который объявляет структуру заданной абстракции, не предоставляя полные реализации методов
- модификатор **abstract** показывает методы, подпадающие под ответственность подкласса, потому что они не имеют реализации в суперклассе
- любой класс, содержащий один или несколько абстрактных методов, тоже должен быть объявлен абстрактным

IThub

```java
public abstract class A {

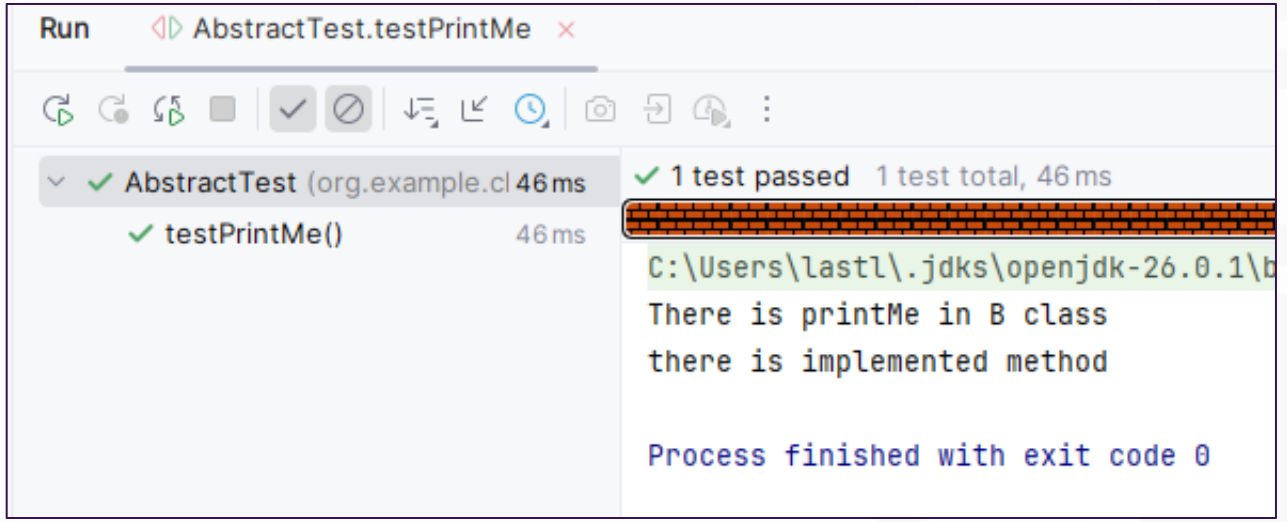
    abstract void printMe();

    public void printMeThere() {
        System.out.println("there is implemented method");
    }
}
```

```java
public class B extends A {

    @Override
    void printMe() {
        System.out.println("There is printMe in B class");
    }
}
```

```java
@Test
public void testPrintMe() {
    B b = new B();
    b.printMe();
    b.printMeThere();
}
```




IThub

# Интерфейсы (interface)

- возможность полностью абстрагировать интерфейс класса от его реализации (можно указать, что класс должен делать, но не как конкретно)
- интерфейсы можно использовать для импортирования общих констант в несколько классов

```
доступ interface имя {
    возвращаемый-тип имя-метода1(список-параметров);
    возвращаемый-тип имя-метода2(список-параметров);
    тип финальное-имя-переменной1 = значение;
    тип финальное-имя-переменной2 = значение;
    // ...
    возвращаемый-тип имя-методаN(список-параметров);
    тип финальное-имя-переменнойN = значение;
}
```

IThub

IThub

```java
public interface Messenger {

    public void sendMessage(String message);

    public String getMessage();
}
```

```java
public class Telegram implements Messenger {

    @Override
    public void sendMessage(String message) {
        System.out.println("Sending message: "
            + message + " to telegram");
    }

    @Override
    public String getMessage() {
        return "Get message from telegram";
    }
}
```

```java
public class Viber implements Messenger {

    @Override
    public void sendMessage(String message) {
        System.out.println("Sending message: "
            +  message + " to viber");
    }

    @Override
    public String getMessage() {
        return "Get message from viber";
    }
}
```

```java
public class MessengerTest {

    @Test
    public void testMessenger() {

        Messenger messenger = new Telegram();

        String message = messenger.getMessage();
        Assertions.assertEquals("Get message from telegram", message);

        Assertions.assertTrue(messenger instanceof Telegram);
        Assertions.assertTrue(messenger instanceof Messenger);

        Assertions.assertFalse(messenger instanceof Viber);
}
```

IThub

# Интерфейсы можно расширять

- с помощью ключевого слова **extends** один интерфейс может быть унаследован от другого
- когда класс реализует интерфейс, унаследованный от другого интерфейса, он должен предоставить реализации для всех методов, требуемых интерфейсами в цепочке наследования

IThub

```java
public interface Messenger {

    public void sendMessage(String message);

    public String getMessage();
}
```

```java
public interface VideoMessenger extends Messenger {

    public void viewMessage();
}
```

```java
public class SomeVideoMessenger implements VideoMessenger{

    @Override
    public void viewMessage() {
        // показывает видео
    }

    @Override
    public void sendMessage(String message) {
        System.out.println("Sending message: " + message + " to some video messenger");
    }

    @Override
    public String getMessage() {
        return "Get message from some video messenger";
    }
}
```

IThub

# СТАНДАРТНЫЕ МЕТОДЫ ИНТЕРФЕЙСА

Стандартный метод интерфейса определяется аналогично определению метода в классе. Основное отличие связано с тем, что объявление предваряется ключевым словом **default**

```
public interface Messenger {

    public void sendMessage(String message);

    public String getMessage();

    public default String getErrorMessage() {
        return "some error occurred";
    }
}
```

```
public class SomeVideoMessenger implements VideoMessenger{
    @Override
    public void viewMessage() {
        // показывает видео
    }

    @Override
    public void sendMessage(String message) {
        System.out.println("Sending message: " + message
                + " to some video messenger");
    }

    @Override
    public String getMessage() {
        return "Get message from some video messenger";
    }

    @Override
    public String getErrorMessage() {
        return "some error with video occurred";
    }
}
```

# Интерфейс для констант – АНТИПАТТЕРН

IThub

```
public interface ConstantsInterface {

int NO = 0;
   int YES = 1;
   int MAYBE = 2;
   int LATER = 3;
   int SOON = 4;
   int NEVER = 5;
}
```

```
public class SomeClass implements ConstantsInterface {

   public final int YES = 100;
}
```

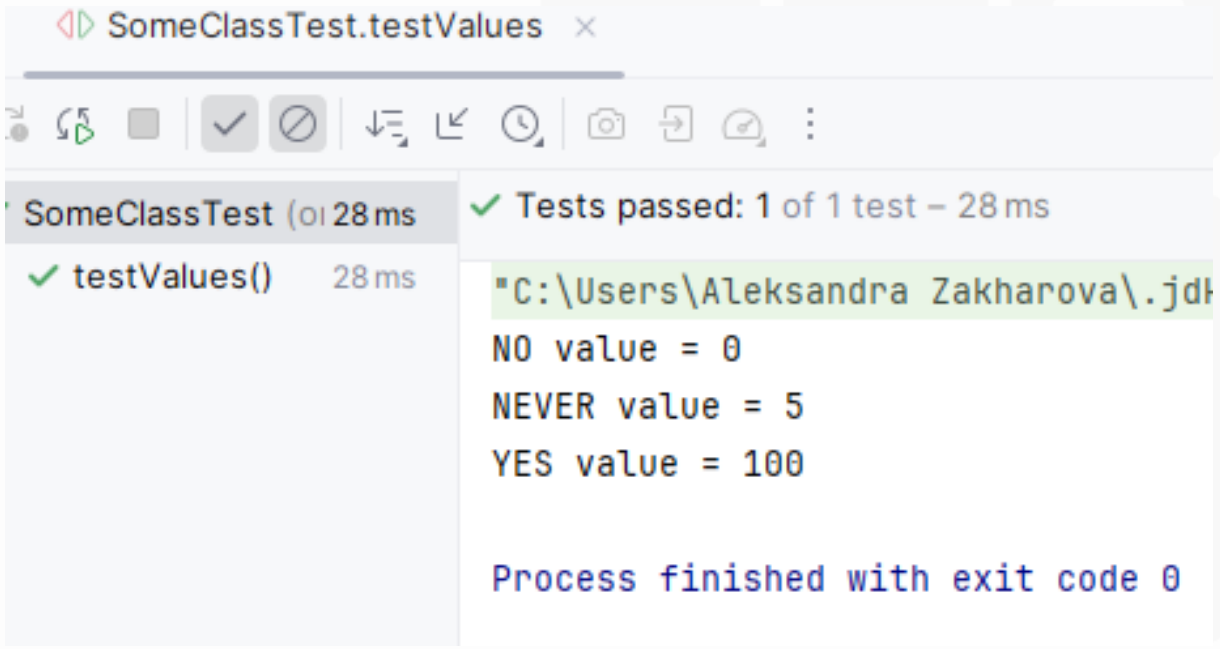
Мы должны использовать интерфейсы для создания контракта на поведение, которое будут обеспечивать наши реализующие классы. Создавая интерфейс, полный константа, мы не определяем никакого поведения.

```
class SomeClassTest {

   @Test
   void testValues() {
      SomeClass someClass = new SomeClass();

      System.out.println("NO value = " + someClass.NO);
      System.out.println("NEVER value = " + someClass.NEVER);

      System.out.println("YES value = " + someClass.YES);
   }
}
```



# Принцип разделения интерфейса (ISP)

## Interface Segregation Principle

**Клиенты не должны вынужденно зависеть от методов, которыми они пользуются**

Если клиенты зависят от методов, которыми они не пользуются, то такие клиенты восприимчивы к изменениям в этих методах. Результатом будет связанность между всеми клиентами.

IThub

```java
public interface Payment {

    void initiatePayments();
    Object status();
    Object[] getPayments();
}
```

```java
public class BankPayment implements Payment {

    @Override
    public void initiatePayments() {
        // TODO some logic
    }

    @Override
    public Object status() {
        // TODO some logic
        return null;
    }

    @Override
    public Object[] getPayments() {
        // TODO some logic
        return new Object[0];
    }
}
```

Есть банковское приложение, которое обрабатывает платежи.

На данный момент реализующему классу BankPayment нужны все методы интерфейса Payment. Таким образом, это не нарушает принцип разделения интерфейса.

IThub

К нам приходит заказчик и говорит, что теперь мы должны обработать платежи по кредиту.

Этот сервис также является разновидностью платежа, но с несколькими дополнительными операциями.

Добавим новые методы в интерфейс Payment и реализуем класс LoanPayment:

```
public interface Payment {

    void initiatePayments();
    Object status();
    Object[] getPayments();

    void initiateLoanSettlement();  // инициировать погашение кредита
    void initiateRePayment();       // инициировать повторный платеж
}
```

IThub

```java
public class BankPayment implements Payment {

    @Override
    public void initiatePayments() {
        // TODO some logic
    }

    @Override
    public Object status() {
        // TODO some logic
        return null;
    }

    @Override
    public Object[] getPayments() {
        // TODO some logic
        return new Object[0];
    }

    @Override
    public void initiateLoanSettlement() {
        // ??
    }

    @Override
    public void initiateRePayment() {
        // ??
    }
}
```

```java
public class LoanPayment implements Payment {

    @Override
    public void initiatePayments() {
        // ??
    }

    @Override
    public Object status() {
        // ??
        return null;
    }

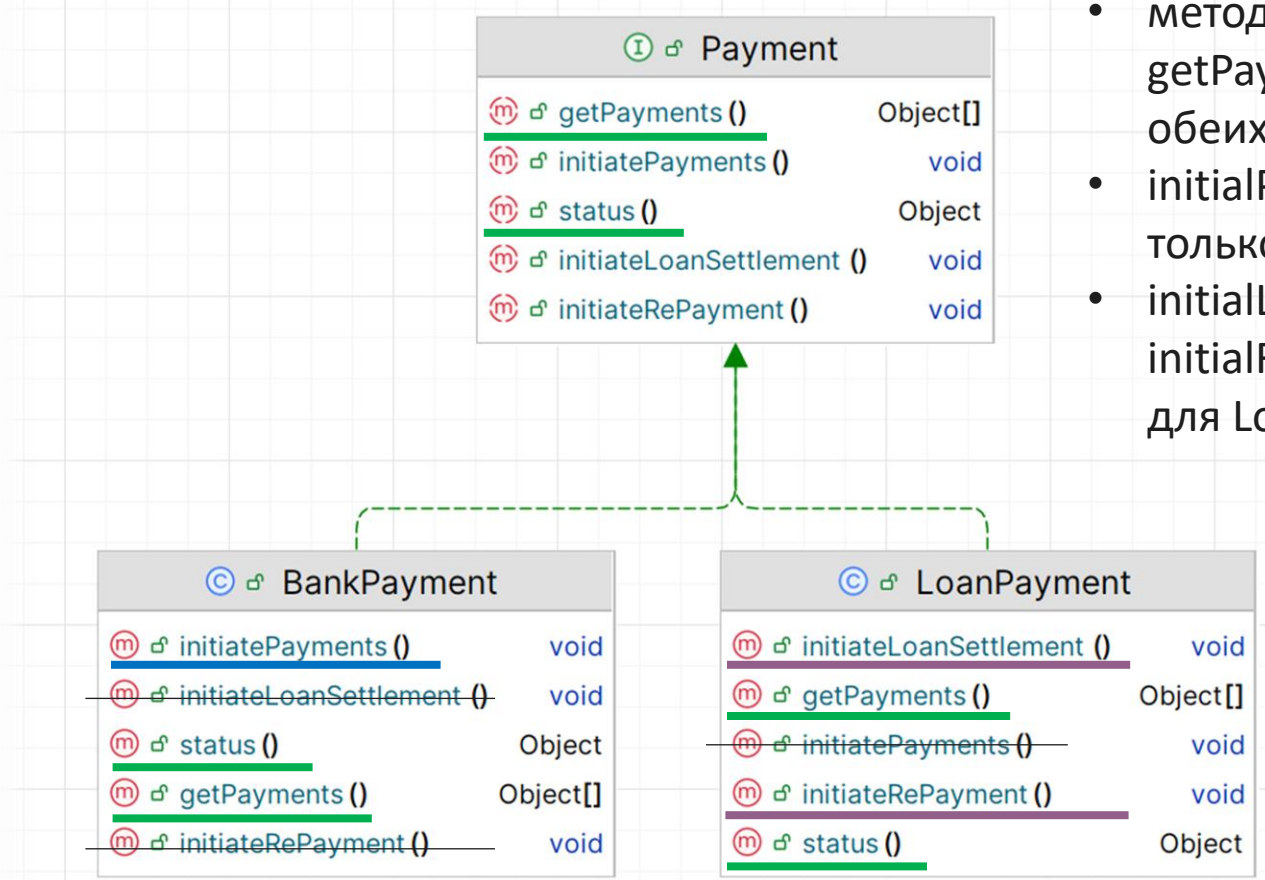
    @Override
    public Object[] getPayments() {
        // ??
        return new Object[0];
    }

    @Override
    public void initiateLoanSettlement() {
        // TODO some logic
    }

    @Override
    public void initiateRePayment() {
        // TODO some logic
    }
}
```

Теперь, после изменения интерфейса Payment и добавления новых методов, все реализующие классы должны реализовать новые методы. Проблема в том, что их реализация нежелательна. В данном случае класс реализации LoanPayment должен реализовать метод initialPayments() без реальной необходимости. Таким образом, принцип нарушается.

IThub




- методы status() и getPayments()  обязательны в обеих реализациях
- initialPayments() требуется только для BankPayment
- initialLoanSettlement() и initialRePayment() — только для LoanPayment

IThub

Теперь интерфейсы не нарушают принцип. Реализации не должны предоставлять пустые методы. Это сохраняет чистоту кода и снижает вероятность ошибок.

```
public interface Payment {

    Object status();
    Object[] getPayments();
}
```

```
public interface iBankPayment extends Payment{

    void initiatePayments();
}
```

```
public interface ILoanPayment extends Payment {

    public void initiateLoanSettlement();

    public void initiateRePayment();
}
```

IThub

```java
public class BankPayment implements iBankPayment {

    @Override
    public void initiatePayments() {
        // TODO some logic
    }

    @Override
    public Object status() {
        return null; // TODO some logic
    }

    @Override
    public Object[] getPayments() {
        return new Object[0]; // TODO some logic
    }
}
```

```java
public class LoanPayment implements iLoanPayment {

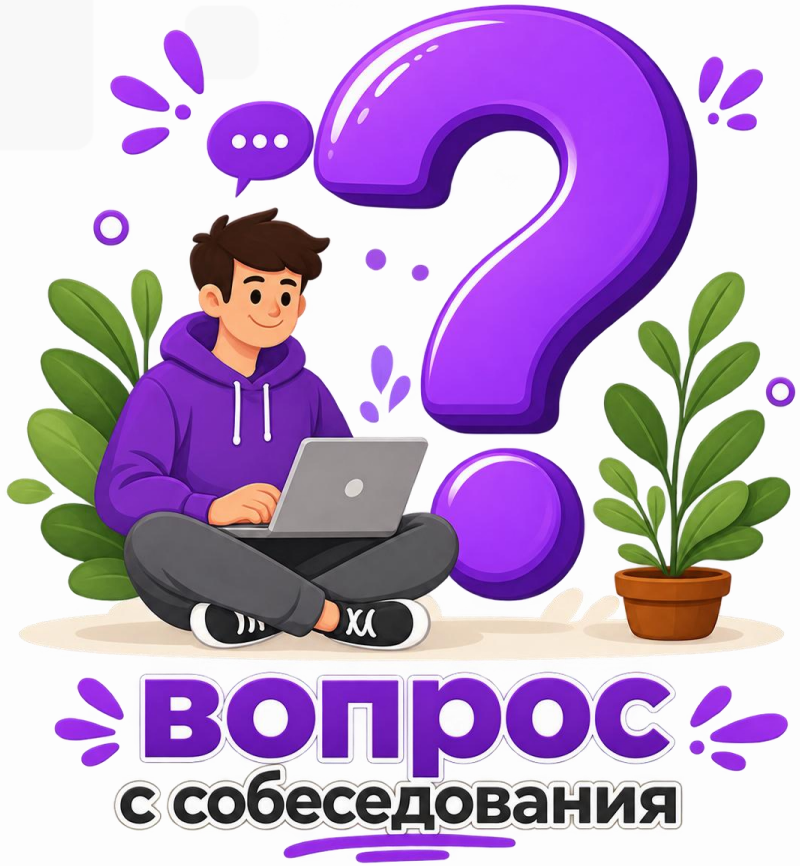
    @Override
    public Object status() {
        return null; // TODO some logic
    }

    @Override
    public Object[] getPayments() {
        return new Object[0]; // TODO some logic
    }

    @Override
    public void initiateLoanSettlement() {
        // TODO some logic
    }

    @Override
    public void initiateRePayment() {
        // TODO some logic
    }
}
```

IThub




**Чем абстрактный класс отличается от интерфейса?**

**В каких случаях следует использовать абстрактный класс, и в каких интерфейс?**

IThub

# Абстрактный класс или интерфейс?

IThub

| Абстрактный класс | Интерфейс |
|---|---|
| **Наследование** | |
| Абстрактный класс может наследоваться только от одного класса и имплементировать любое количество интерфейсов | Интерфейс не может наследоваться от классов, но может имплементировать любое количество интерфейсов |
| **Абстрактные методы** | |
| Абстрактный класс может содержать абстрактные методы. Но может и не содержать их вообще | Все не статические и не default методы интерфейса – абстрактные – не содержат реализации. Интерфейс может не содержать никаких методов вообще |
| **Методы с реализацией** | |
| Абстрактный класс может содержать методы с реализацией | Интерфейс может содержать методы по умолчанию |
| **Данные** | |
| Никаких ограничений | Интерфейс содержит только public final static данные |
| **Создание объекта** | |
| Нельзя создать объект абстрактного класса | Нельзя создать объект интерфейса |

# Когда использовать интерфейс

- Если проблема должна быть решена с использованием множественного наследования и состоит из **различных** иерархий **классов**
- Когда **несвязанные классы** реализуют наш интерфейс
- Когда функциональные возможности приложения должны быть определены как контракт, но не важно, кто реализует поведение, т.е. сторонние поставщики должны реализовать его полностью

# Когда использовать абстрактный класс

- При попытке использовать концепцию наследования в коде (совместное использование кода многими **связанными классами**), предоставляя общие методы базового класса, которые переопределяют подклассы. Класс-наследник только расширяет функциональность абстрактного класса.
- Если у нас есть определенные требования и только частичные детали реализации.
- Когда расширяющие абстрактные классы, имеют несколько общих полей или методов (которые требуют непубличных модификаторов).
- Если кто-то хочет иметь нефинальные или нестатичные методы для изменения состояний объекта.

# Задание

IThub

Создайте программу, которая моделирует разных игровых персонажей.
В игре есть разные типы персонажей: воин, маг и лекарь.
У всех есть общие характеристики, но каждый персонаж имеет свои способности.

**1. Создайте абстрактный класс Character**
Он должен содержать поля:
- name — имя персонажа
- health — количество здоровья

Поля должны быть защищены от прямого изменения.
Добавить конструктор с 2 аргументами, метод showInfo(), который выводит информацию и абстрактный метод public abstract void attack()

**2. Создайте интерфейс MagicUser**
Интерфейс описывает способность использовать магию.
Добавьте метод void useMagic()

**3. Создайте классы-наследники**
⚔️ Класс Warrior наследуется от Character
Добавьте поле: «оружие».
Реализуйте метод:attack() – воин Артур атакует мечом
🧙‍♂️ Класс Wizard наследуется от Character и реализует интерфейс MagicUser
Добавьте поле заклинание. Реализуйте методы:
- attack(): Маг Мерлин атакует заклинанием
- useMagic(): Мерлин использует огненный шар

🧑‍⚕️ Класс Healer наследуется от Character и реализует интерфейс MagicUser
Добавьте поле: «уровень лечения».
Реализуйте методы:
- attack(): Лекарь атакует посохом
- useMagic(): Лекарь восстанавливает здоровье союзника

IThub

**4. В классе Main**

Создайте массив персонажей:

Character[] team = { new Warrior(...), new Wizard(...), new Healer(...)};

Переберите массив и вызовите методы:

- showInfo()
- attack()

**5. Работа с интерфейсом**

Создайте массив: MagicUser[] magicTeam

Добавьте туда только персонажей, которые умеют использовать магию:

- Wizard
- Healer

Вызовите: useMagic();

**6. Добавьте новый класс:**

🏹 Archer наследуется от Character

имеет своё оружие

реализует метод attack(): Лучник стреляет из лука

Добавьте его в команду.